

bubble sort

[4, 1, 5, 2, 3]

(n-1) iteration loop

adjacent element compare

largest element push

using swap

[4, 1, 5, 2, 3]

↑ Pair.

in adjacent element compare 4, 1, 5, 2, 3

when first element is large than second push

4, 1, swap [1, 4, 5, 2, 3]

4, 5 swap [1, 4, 5, 2, 3] not change because first element is small.

5, 2 swap [1, 4, 2, 5, 3] if (A[j] > A[j+1])

5, 3 compare swap. [1, 4, 2, 3, 5] swap

first element push in last finally largest

code

```
for (i = 0; i < n - 1; i++) {  
    for (j = 0; j < n - 1 - i; j++) {  
        if (arr[j] > A[j + 1])  
            swap(A[j], A[j + 1])  
    }  
    A[j + 1] means next  
    A[j - 1] means previous
```

$O(n^2)$ time complexity

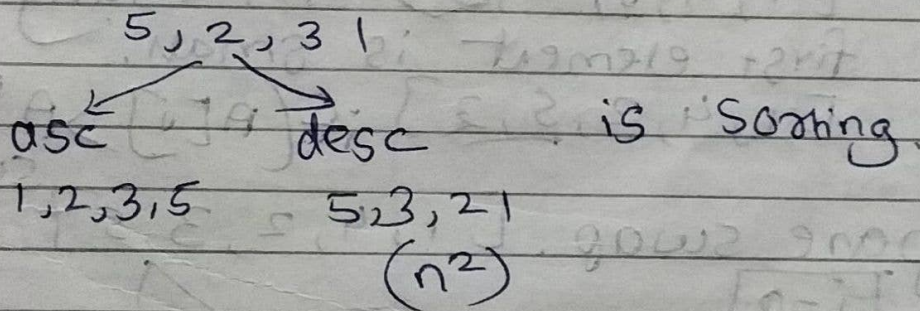
If key is found at very first position in only one no. of comparison as a best case $O(1)$

If either key is found at last position or does not exist maxi. n no. of comp
worst case $O(n)$

found in between position average case
 $O(n)$

Sorting Algorithms.

Sorting = arrange data in particular order.



bubble sort

Best case = if array element already sorted
 $O(n)$

worst case = $O(n^2)$

Average case = $O(n^2)$

Second iteration

i value

- $[1, 4, 2, 3, 5]$
- $[1, 4, 2, 3, 5]$
- $[1, 2, 4, 3, 5]$
- $[1, 2, 3, 4, 5]$

Two largest element

$n-1$

third iteration

$i=02$

- $1, 2, 3, 4, 5$
 - $1, 2, (3, 4, 5)$
 - $1, 2, 3, 4, 5$
- three largest

four iteration

$i=43$

- $1, 2, 3, 4, 5$
- $1, 2, 3, 4, 5$

$n=5$, run $n-1$ so run 4 time

when run first time $i=0, i=1, i=2, i=3$

- $i=0$ 4 comparison line
- $i=1$ 3 comparison
- $i=2$ 2
- $i=3$ 1

$n-i-1$	
$n-i-1$	$5-0-1=4$
$n-i-1$	5,
$n-i-1$	

Best case = $O(n^2)$
 worst case = $O(n^2)$
 Average case = $O(n^2)$ Same all.

Selection Sort

- $[4, 1, 5, 2, 3]$

sorted

unsorted

Imagine two part sorted, unsorted

Pick element smallest element from unsorted to sorted

$n-1$ iteration

$n-1$ smallest

- $[4, 1, 5, 2, 3]$ is unsorted Pick one small element
- $[1, 4, 5, 2, 3]$
- sorted | unsorted

1, 4, 5, 2, 3
sorted ← 1 unsorted pick small two

1, 2 4, 5, 2, 3
sorted ← unsorted pick small element
which is 2

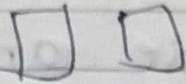
1, 2, 3 4, 5, pick 4
1, 2, 3, 4, 5

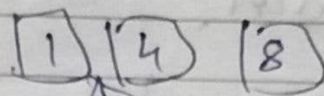
code 1st pass: smallest element pick
and swap with first elem

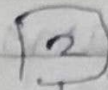
second pass: 2nd smallest element pick
— 2nd position

third pass: smallest

Insertion Sort

e.g.  card game





insert

- Insertionsort algorithm is an efficient algorithm for smallest input size array work efficiently for ~~an~~ already sorted input case time $O(n)$ in best

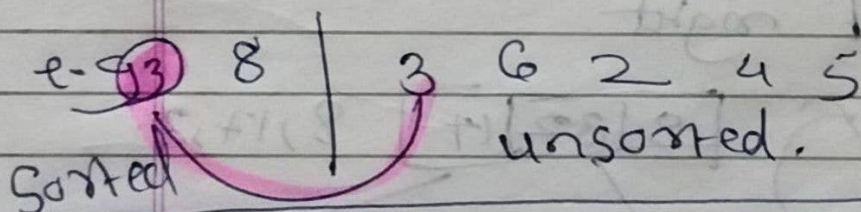
Best case = $O(n)$ already sorted manner
worse case = $O(n^2)$
Average case $O(n^2)$

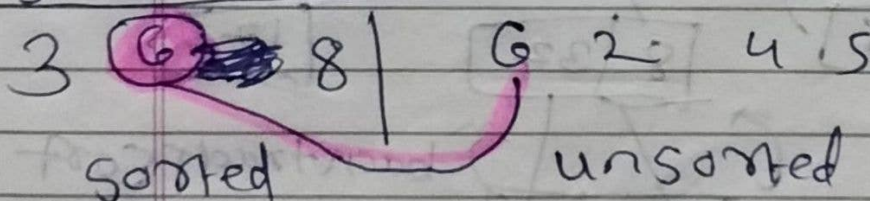
code

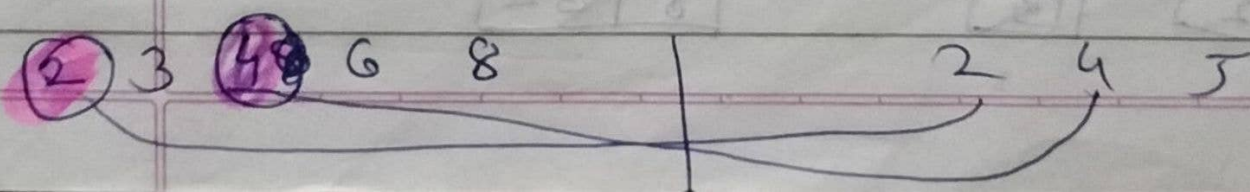
Sorted / unsorted array

Take one element, iterate through sorted find correct position

not select minimum in ~~selection~~ insertion

e.g. 





Merge Sort

This algorithm follows divide and conquer approach.

Best case = $O(n \log n)$
Worst case = $O(n \log n)$
Average case = $O(n \log n)$

Space complexity

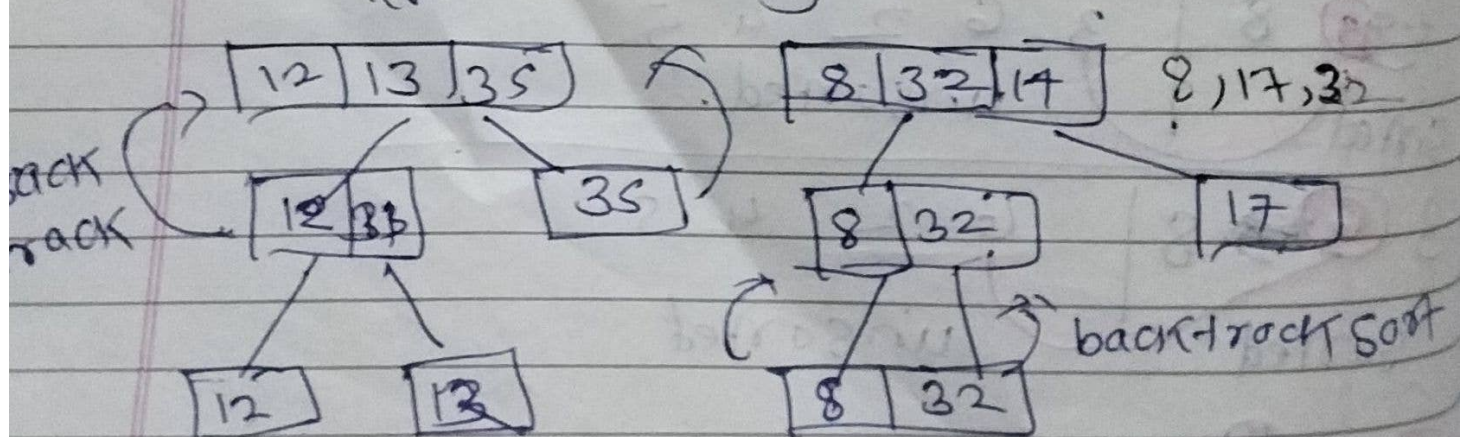
This algorithm works fine for even & odd.

This algorithm takes extra space complexity more.

Recursion base Algorithm
merge sort used to linear structure
array, vector increasing, decreasing
sort, ascending, descending

1) Divide array in Two equal parts

12, 31, 38, 5 | 8, 32, 17
left right



Quick Sort

In this algorithm basic logic is partitioning.

This algorithm follows divide-conquer

Best case of quick sort = $O(n \log n)$

Worst case $O(n^2)$

Average case $O(n \log n)$

* Quick Sort algorithm is an efficient sorting algorithm for larger input size array.

arr = [5, 2, 6, 4, 1, 3]

Pivot & partition

special index

in place sorting

choose first, last, mid also random element as pivot
last element

[5 | 2 | 6 | 4 | 1 | 3]

↑ pivot choose

└──┬──┐ pivot └──┬──┐
≤ greater
less

2, 1, [3], 5, 6, 4

left half

right

insertion sort
~~public class Main~~

public static void main(String[] args) {

Scanner sc = new Scanner(System.in)

int n = sc.nextInt();

int arr[] = new int [n]

for(int i=0; i<n; i++) {

arr[i] = sc.nextInt();

for(int i=1; i<n; i++) {

int current = arr[i]

int j = i-1;

while(j >= 0 && arr[j] > current) {

// keep swapping

arr[j+1] = arr[j];

j--

}

arr[j+1] = current;

}

printArray(arr);

}

dry run

How to find correct position insertion

Sort: 2 5 7 9 13 15
 0 1 2 3 4 5
 7 compare 7 7 7 7 7
 8 13 15 15 15
 2 3 4 5

Sort: [2 5 7 9 13 15]

15 and 7 compare

Swap

13 and 7 compare swap

9 and 7 compare swap

5 and 7 compare correct order found.

e.g cards

2 3 6 8
 1 2 3

$j = 4$
 4 5
 4 5
 4 5

j point element

whose insertion index 4

$a[j] < a[j-1]$ compare

$j--$

4 index become 3

$j = 3$ index value 4

insertion sort

Insertion Sort Sorting algorithm in which we take one element at time insert it into correct position in already sorted part of the array.

$arr[j+1] = arr[j]$...

if previous element greater ^{shift right position}

* The loop start from index 1 because first element considered already sorted.

* Store current element in variable current

* j point to the element just before current element, $int j = i - 1$;
 $while(j \geq 0 \ \&\& \ arr[j] > current)$

This loop check wheather j is within array whether previous element is greater than current element.

```
import java.util.*;
```

```
public static void printArray (int arr[])
```

```
{  
    for (int i=0; i < n; i++) {
```

```
        System.out.println(arr[i] + " ");  
    }
```

```
}  
System.out.println();
```

public class Selectionsort

public static void main (String [] args)

~~int arr~~

Scanner sc = new Scanner (System.in);

int n = sc.nextInt(); // size

int arr[] = new int [n]

enter size input

enter array element
create array

for (int i = 0; i < n; i++) // array element
arr[i] = sc.nextInt();

for (int i = 0; i < n; i++) {

arr[i] =
int min = i

i = current position
j = used for swap
min = store index
of smallest element
temp = used for swap

for (int j = i + 1; j < n; j++) {

if (arr[j] < arr[min]) {
min = j;

// store minimum

}
int temp = arr[j];
arr[j] = arr[min];
arr[min] = temp;

// sorted array print

for (int i = 0; i < n; i++) {
System.out.print (arr[i] + " ");

sc.close();

public class Selectionsort {

public static void main (String [] args)

int arr

Scanner sc = new Scanner (System.in);

enter size input

int n = sc.nextInt(); // size

int arr[] = new int [n]

enter array element
create array

for (int i = 0; i < n; i++) // array element
arr[i] = sc.nextInt();

for (int i = 0; i < n; i++) {

arr[i] =
int min = i

i = current position
j = used for swap
min = store index
of smallest element
temp = used for swap

for (int j = i + 1; j < n; j++) {

if (arr[j] < arr[min]) {
min = j;

// store minimum

}

int temp = arr[j];

arr[j] = arr[min];

arr[min] = temp;

// sorted array print

}
for (int i = 0; i < n; i++) {

System.out.print (arr[i] + " ");

sc.close();

public class Selectionsort {

public static void main(String[] args)

int arr

Scanner sc = new Scanner(System.in);

int n = sc.nextInt(); // size

int arr[] = new int[n]

for (int i = 0; i < n; i++) // array element
arr[i] = sc.nextInt();

for (int i = 0; i < n; i++) {

arr[i] =
int min = i

for (int j = i + 1; j < n; j++) {

if (arr[j] < arr[min]) {
min = j;

// store minimum

}
int temp = arr[j];
arr[j] = arr[min];
arr[min] = temp;

}
for (int i = 0; i < n; i++) {
System.out.print(arr[i] + " ");

sc.close();

enter input
size

enter array
element
create array

i = current position
j = used for swap
min = store index
of smallest element
temp = used for swap

// sorted array
print

Selection Sort is sorting technique in which we find smallest element from unsorted part of the array and place it in correct position.

Sorted part increases by one element after every pass.

This process continues until all elements are placed in their correct position and complete array becomes sorted.

Best case $O(n)^2$ Space complexity $O(1)$
 Average $O(n)^2$
 Worst $O(n)^2$

bubble sort	Selection
compare adjacent element	find minimum element
Swap adjacent element	swap minimum with current position
use $j, j+1$	use min and j
multiple swaps per pass possible	At most one swap per pass.

```
public class BubbleSort {
```

```
    public static void main(String[] args) {
```

```
        int[] arr = {4, 1, 5, 2, 3};
```

```
        for (int i = 0; i < arr.length - 1; i++) {
```

```
            for (int j = 0; j < arr.length - 1 - i; j++) {
```

```
                if (arr[j] > arr[j+1]) {
```

```
                    int temp = arr[j];
```

```
                    arr[j] = arr[j+1];
```

```
                    arr[j+1] = temp;
```

```
                }
```

```
            }
```

```
        System.out.println(arr[i] +
```

```
            " ");
```

```
    }
```

```
}
```

Outer loop, control number of passes
e.g 5 element, maximum 4 passes.

Inner loop, compare adjacent element
after every pass, large element
move to end.

Sorting

Page No.
Date

$O(n)$ if array sorted.
 $O(n^2)$ worst case
 $O(n^2)$ Average case

* byte b = (byte) 128
output - 128

* byte b = (byte) 129
output - 127

* byte b = (byte) 130
output - 126

we use $n-1-i$ because
every pass, one element is placed at correct position at end.
we reduce number of comparison

$n=5$

$j < 5-1 = 0$

$j < 4 = 4$ comparison

$j < 3 = 3$ comparison

bubble sort is sorting technique in which we compare two adjacent elements.

If the first element is greater than second element, we swap them.

We repeat this process until the array is sorted.

(a) 4 1 5 2 3

(a) 4 1 swap 1 4 5 2 3

5 2 swap

when first element is larger than second element

$a[j] > a[j+1]$